# CSC_51053 Lab: Query Optimization

Ioana Manolescu, Madhulika Mohanty

November 7, 2025

## 1 Setting up the movie database

We use a *movie database*, which is a version of the IMDB web site[1]. We provide you a link to a database dump in Moodle.

**To load the IMDB database in your Postgres server:**

- Unzip the file, you will get a file called `imdb.sql`;
- Launch a Postgres server (`pg_ctl start`), then connect to it via the `psql` client;
- Do this in the client: `create database imdb;` then exit the client.
- You will need to **load the dump and measure how long it took**:
  - On Unix/Linux/Mac, or on Windows using WSL (Windows Subsystem for Linux): `time psql imdb < imdb.sql` should both load the dump, and measure the time.
  - On Windows using Powershell, `Measure-Command { psql imdb < imdb.sql }`

  Write the duration in a **experiment documentation file** where you will add more pieces of information. **Announce it** so that we may compare different durations to see the impact of hardware/software configurations.

Connecting to the IMDB database and typing `\dt` (describe tables) should show you 25 tables. Their numbers of tuples go from more than 50 million tuples, to just 4 tuples. More information about the schema is available in a separate document in Moodle (but it's not really needed).

## 2 Documenting parameters that impact on the performance of your database

**Find these information about your laptop and write them in the documentation file:**

- What type of disk(s) your laptop has: the brand, the model, the size, the transfer rate if available (at what speed the data can get from disk into memory).
- How much memory the Postgres server has. For this, in the `psql` client, type: `show shared_buffers`.
- The version of your Postgres server: `select version()`;

## 3 Queries and their performance

We will inspect the performance of two queries on the IMDB database.

[1] `https://www.imdb.com/interfaces/`

### 3.1 All (movie title, actor name) pairs

The following query computes the pairs of (movie title, actor name) such that the actor played in the movie:

```
select distinct m.title, n.name
from movie m, cast_info c, person n, role_type r
where n.id = c.person_id
and c.movie_id = m.id
and c.role_id = r.id
and (r.role='actor' or r.role = 'actress')
```

Because the table names contain underscores (_), copy-pasting the query from the PDF may not work. You can find it in the file "queries.sql", in Moodle.

**Inspecting plans and analyzing their performance**

1. In the `psql` client, launch the timer: `\timing`.
2. Then, still in the `psql` client, ask: `explain analyze verbose query1` (replace query1 by the actual query). Copy the output of this command, in a file.
3. Go to https://explain.dalibo.com and paste this output. Inspect the plan visualization (click on the various operators to see more details), and find out:
   (a) What are the two slowest operators? Add this info (and all the subsequent one) to your experiment documentation file.
   (b) What is the most selective operator?
   (c) Now select the "rows" view at the bottom right of the visualization. What are the two operators returning most rows?
   (d) Now select the "cost" view at the bottom right of the visualization. This shows the cost as *estimated by the optimizer*. Which operator was considered to have the highest cost (the most expensive)? Compare with the slowest (that you can see again by clicking "duration"). Write the name, duration, and cost of these operators (the one with the highest estimated cost; the slowest). Are they the same?
   (e) Is there an operator with an important estimation error? An estimation error is the difference between the number of rows produced by the operator, and the optimizer's estimation of this number of rows. An error is important if the difference is large (in absolute terms); also, because estimation errors propagate up in the tree, an error made in a child operator will likely impact parent operators, too.
4. Ask in the client: `select setting from pg_settings where name = 'data_directory'`. Write down the resulting directory, call it PGDIR.
5. Exit all (**ALL!**) the `psql` clients.
6. Stop the server: `pg_ctl stop -D PGDIR` (replace PGDIR with its value).
7. Go edit the file `PGDIR/postgresql.conf`: modify the value of `shared_buffers`. You can increase it, e.g., double it, or halve it to simulate a more constrained situation. Save the file.
8. Relaunch the server `pg_ctl start -D PGDIR`.
9. In a new client, re-run the `explain analyze verbose query1`. Compare the plan with the one previously obtained. Is there a difference with the previous plan?

### 3.2 King movies

This query finds all 2015 movies whose titles mention a king:

```
select m.title, p.name
from movie m, cast_info c, person p, role_type r
where p.id=c.person_id and c.movie_id=m.id and c.role_id=r.id and
r.role='actor' and m.title like 'King%' and m.production_year=2015;
```

**Improving the performance of a query**

1. Get the plan for this query (`explain analyze verbose query2`);
2. What is the most inefficient operator?
3. Propose (a) an index and (b) a multi-attribute statistic that can improve query performance.
   **Hint** The syntax for asking Postgres to compute the most frequent *combinations of values* of two attributes is:
   `create statistics s1(mcv) on att1, att2 from table;`
4. Implement these measures, get the resulting plan, inspect it with Dalibo. Did performance improve?